Assignment 2
Digit Recognizer

Estelle Arnander

Kaggle username: estellearnander03

1. Objective

I took the two reference points from the instructions as targets: at least 0.97 validation accuracy for the MLP baseline and at least 0.99 for the CNN. Beyond the required components, I designed the experiments to answer 3 questions:

1. How much does the CNN improve on the MLP?
2. Is that improvement due to convolutions or only to the CNN having more parameters?
3. Which improvement (BatchNorm and dropout, data augmentation, a learning-rate schedule) reduces the CNN's overfitting, measured one change at a time?

2. Data preparation

The data passed every check. I split it 90/10 with stratification and normalised it with statistics from the training split only.

Checks. Before training anything, `check_data` confirms that both files have the expected shape (42,000 x 785 and 28,000 x 784). It also confirms that `label` is the first column of the training file, that there are no missing values, that labels lie between 0 and 9 and that pixels lie between 0 and 255. These checks protect the assumptions the rest of the code relies on. The model has 10 outputs, and `CrossEntropyLoss` uses each label as the index of the correct output. A label outside 0 to 9 would point at an output that does not exist and crash training. The preprocessing divides the pixels by 255 to bring them into [0, 1]. If the pixels were on another scale, for example already between 0 and 1, every image would become almost black and the normalisation statistics would be wrong. A wrong column order would not raise an error during training but the model would learn from the wrong labels, which is why I check it explicitly.

Train, validation and test sets. I kept 10% of the labelled images (4,200) as the hold-out validation set and trained on the other 90%. The validation set is used to detect overfitting, to pick the best epoch and to compare models. I did not cut a separate internal test set, because it would cost another 4,200 training images and give an estimate no more precise than validation.

The split is 90/10 rather than the 80/20. A larger validation set gives a more precise score, but with 42,000 images, 10% is already 4,200 images. Going to 20% would only narrow the uncertainty from about +- 0.3 to +-0.2 points while removing another 4,200 training images. The split is stratified, so every digit has the same share in both parts. It also uses a fixed seed, so every model is evaluated on exactly the same images.

I used a single hold-out split instead of 5-fold cross validation. Cross validation trains five models, each validated on a different 1/5 part of the data, and averages their scores. Every image is then used for validation once, so that the estimate rests on 42,000 images instead of 4,200 and is much more precise. The cost is five times the training time. The experiments on the training split would take about 85 minutes instead of 17 per run, and the project needed several runs. Section 4 shows what this imprecision means for comparing the models.

Class balance. The classes are roughly balanced. The most common digit is 1, with 4,216 training images. The least common is 5, with 3,415. That is a ratio of about 1.2. Accuracy can be misleading when one class dominates, because a model can then score well by mostly predicting that class. With an imbalance this small, accuracy is a fair metric, so I did not weight the classes in the loss.

Normalisation. I divided the pixels by 255 and then standardised them with the mean (0.131) and standard deviation (0.3085) of the training split. Standardising centres the inputs around zero on a similar scale, which makes gradient-based optimisation easierI computed the statistics on the training split only. If they included the validation images, information from the validation set would leak into the preprocessing, and the validation accuracy would be slightly too optimistic. After normalisation, the black background has the value −0.42 and the brightest strokes 2.82.

Dataset and DataLoader. A custom `DigitDataset` implements `__len__` and `__getitem__` (lecture 7, slide 4). It reshapes each row of 784 pixels into an image of shape [1, 28, 28]. That is one channel of 28 rows and 28 columns. Pixels are stored as float32 because the model's weights are float32. Labels are stored as int64 because `CrossEntropyLoss` expects class indices of that type. The DataLoader groups the images into batches of shape [128, 1, 28, 28]. The training loader shuffles with a seeded generator, so the shuffle order is the same on every run.

Order of operations. Augmentation (section 3) runs after scaling to [0, 1] and before normalisation. Rotating or shifting an image leaves empty corners, and these are filled with 0. Before normalisation, 0 means black background. After normalisation, black is −0.42, so a fill of 0 would create grey patches that never appear in real digits.

Sample digits and the class distribution are shown in the notebook and saved in `results/` (`sample_digits.png`, `class_distribution.png`).

3. Models and architectures

I trained seven models. They are an MLP baseline, a plain CNN and an improved CNN, which I trained in three versions. The first has no augmentation, the second adds augmentation and the third also adds a learning-rate schedule. Two further control CNNs test whether model size explains the CNN's advantage. Every model takes batches of shape [B, 1, 28, 28] and returns 10 logits. I did not add a softmax at the end, because `CrossEntropyLoss` applies it internally and expects raw logits (lecture 5, slide 25; Session 9 notebook, section 5).

Model	Architecture	Parameters
MLP	Flatten → 784 → 256 → 128 → 10, ReLU, dropout 0.2	235,146

Model	Architecture	Parameters
SimpleCNN	2 blocks of Conv 3x3 (32, then 64 filters) → ReLU → MaxPool 2, then Flatten → 128 → 10	421,642
SimpleCNN-GAP (control)	Same 2 blocks, then global average pooling → 10	19,466
SimpleCNN-64 (control)	SimpleCNN with a 64-unit hidden layer instead of 128	220,234
ImprovedCNN	SimpleCNN + BatchNorm after each convolution, dropout 0.25 after each block and 0.5 in the head	421,738

MLP baseline. The MLP follows the structure from lecture 5. It flattens each image into a vector of 784 values. Flattening throws away the layout of the image. The MLP does not know which pixels are neighbours, so it has to learn every spatial pattern from scratch, and it has to learn the same pattern again for every position where it can appearI used dropout 0.2 instead of the 0.5 in the lecture example. The network only has two small hidden layers, and I expected that switching off half of their neurons at every step would remove too much of its capacity. I did not test 0.5, so this choice is a reasoned default and not a measured result.

Plain CNN. The CNN keeps the image as a 28 x 28 grid. It has two convolutional blocks. Each block is a 3 x 3 convolution, a ReLU and a 2 x 2 max pooling. A 3 x 3 filter only looks at a pixel and its direct neighbours, which matches how strokes are built from nearby pixels. The same filter is applied at every position, so a stroke the network learns to detect in one corner is also recognised in the middle. These two assumptions, locality and translation invariance, are the CNN's inductive bias. They shrink the space of functions the model has to search, so it needs fewer examples to find a good one. Padding 1 keeps the height and width unchanged after each convolution, and each pooling halves them. The shape therefore goes from [B, 1, 28, 28] to [B, 32, 14, 14] and then to [B, 64, 7, 7] (lecture 8, slides 31 and 35). I kept this model plain, without BatchNorm, dropout or augmentation, so that each later improvement can be measured against it.

Control models. The plain CNN has 421,642 parameters, almost twice as many as the MLP. Of these, 401,536 sit in its first dense layer. So the CNN could beat the MLP simply because it is bigger, and not because of its convolutions. I tested this with two control models. The first replaces the dense head with global average pooling which averages each 7 x 7 feature map into a single number. This removes about 95% of the parameters. The second keeps the CNN unchanged and only shrinks its hidden layer from 128 to 64 units. That gives 220,234 parameters, fewer than the MLP has.

Improved CNN. The improved CNN adds BatchNorm and dropout to the plain CNN and changes nothing else. BatchNorm normalises the output of each convolution using the statistics of the current batch. This keeps the scale of the activations stable from one layer to the next, which makes the gradients more stable. The noise in the batch statistics also gives a slight regularising effect. Dropout switches off a random 25% of the activations after each convolutional block and 50% in the head, at every training step. The network then cannot memorise training images through a few specific neurons and has to learn features that generalise. Both layers behave differently during training and evaluation. In evaluation, dropout is switched off and BatchNorm uses running averages instead of the batch statistics.

That is why the training code calls `model.train()` and `model.eval()`. The convolutions in this model have no bias term, because the shift that BatchNorm learns does the same job.

Data augmentation. Every time a training image is used, it is randomly rotated by up to +- 10 degrees, shifted by up to 10% and zoomed between 90% and 110%. The network never sees exactly the same image twice, so it cannot memorise individual images. These transformations copy the natural variation in how people write digits. I left out flips because a mirrored digit is not the same digit. I kept the rotations small because a large rotation can turn a 6 into a 9. Validation and test images are never augmented, so their scores stay comparable.

Learning-rate schedule. A cosine schedule lowers the learning rate smoothly from 0.001 towards 0 over the epochs. It is the same scheduler as in the Session 12 notebook. Large steps at the start make fast progress. Small steps at the end let the weights settle in a minimum instead of jumping around it.

Why not transfer learning. I considered transfer learning and rejected it, using the decision table in the Session 12 notebook. For a task in a different domain with plenty of data, the table recommends full fine-tuning or training from scratch. Both conditions hold here. Pretrained models such as ResNet were trained on 224 x 224 colour photographs of objects, which is a very different domain from 28 x 28 grayscale pen strokes. This dataset also has about 4,200 images per class, which is within the 1,000 to 5,000 per class that lecture 13 gives as enough to train from scratch. Using a pretrained ResNet would also require enlarging every image eight times and copying it into three channels, with no expected gain.

4. Training and validation

All models were trained the same way and compared on the same 4,200 validation images. Differences smaller than about 0.3 points, or 13 images, cannot be told apart from noise.

Training setup. Every model is trained with Adam (learning rate 0.001), cross-entropy loss on the logits and batches of 128 images. Adam adapts the step size for each parameter and adds momentum, so it needs less tuning of the learning rate than plain SGD. It clears the old gradients, runs the forward pass, computes the loss, backpropagates and updates the weights. PyTorch adds new gradients to the old ones by default, so without the first step the update would mix gradients from different batches. The models without regularisation train for 15 epochs, and B, C and D for 30. Augmentation makes the training images harder, so these models need more epochs. I gave B 30 epochs as well, so that training length is not a second difference between B and the augmented models C and D. On the Mac's GPU, one run took about 13 seconds for the MLP, 70 to 80 seconds for the plain CNN and 155 to 170 seconds for model D.

Choosing the epoch. After every epoch, the model is evaluated on the validation set. At the end, the weights of the epoch with the highest validation accuracy are restored. This works like early stopping without cutting the training short. I select on accuracy and not on loss because accuracy is the Kaggle metric. The two can disagree. For the MLP, validation loss was lowest at epoch 8 and accuracy highest at epoch 12. Loss also penalises correct answers given with low confidence, while accuracy only counts right or wrong. Selecting the epoch on the validation set makes the reported validation accuracy slightly too optimistic, because the best of 15 or 30 noisy values is picked. With so few candidates the bias is small, but it is one more reason to be careful with small differences.

How precise is one validation score? An accuracy p measured on n images has a standard error of

$$SE = \sqrt{\frac{p\,(1-p)}{n}}$$

With p = 0.99 and n = 4,200, the standard error is about 0.15 points, so the 95% interval is about +-0.3 points, or about +-13 images. The training logs confirm this. The MLP's validation accuracy moved from 0.9786 to 0.9764 to 0.9776 over three epochs without any real change in the model. So I only treat a difference between models as real when it is clearly larger than 13 images. MLP to CNN (90 to 44 errors) clearly passes. Plain CNN to B (44 to 30 errors) is just above the threshold, and the lowest validation loss falling from 0.0433 to 0.0295 supports it. B, C and D (30, 31 and 28 errors) cannot be separated. All models are evaluated on the same images, so a paired test such as McNemar's test would be more precise than this rule of thumb. I did not run one.

Reproducibility. Every random step uses a fixed seed. That covers the split (42), the weight initialisation, the augmentation and the shuffling. Each model also gets its own freshly created training DataLoader with a seeded generator. This last point came from a bug I found during the project. In an earlier version, all models shared one DataLoader. Its random generator moves forward every time the loader is used, so each model's shuffle order depended on how many epochs the models before it had trained. When I added a control model before model B, B's result changed from 0.9929 to 0.9933 with identical code and seeds. I fixed it by giving each model its own loader and re-ran every experiment. In the final run of the notebook after a kernel restart, every result repeated exactly. The submission file and the five model weight files were byte-identical to the ones from the earlier run, and only the training times differed by a few seconds. The Mac's GPU does not guarantee bit-identical results on other hardware, so the weights of the five final models are saved in `results/models/`. The submitted predictions can be reproduced exactly from these files.

5. Results and Kaggle score

The final submission scored **0.99232**. It is the average of five copies of model D, each trained on all 42,000 labelled images with a different seed.

Validation results. All models were evaluated on the same 4,200 validation images.

Model	Parameters	Validation accuracy	Errors (of 4,200)
MLP baseline	235,146	0.9786	90
SimpleCNN	421,642	0.9895	44
SimpleCNN-GAP (control)	19,466	0.9436	237
SimpleCNN-64 (size-matched control)	220,234	0.9910	38
B: + BatchNorm + dropout	421,738	0.9929	30

Model	Parameters	Validation accuracy	Errors (of 4,200)
C: B + augmentation	421,738	0.9926	31
D: C + cosine schedule	421,738	0.9933	28

Both targets from the instructions are met. The MLP reaches 0.9786, above 0.97. The plain CNN reaches 0.9895, two images short of 0.99, and every regularised CNN is above 0.99, as is the Kaggle score.

MLP versus CNN. The plain CNN halves the MLP's errors, from 90 to 44. The size-matched control has fewer parameters than the MLP and still makes only 38 errors. Halving its dense layer did not hurt it either, since 38 against 44 errors is inside the noise band. The advantage therefore comes from the convolutions and not from model size. The GAP control failed with 237 errors. Its training accuracy stayed at 0.943, so it was underfitting. With only two convolutional blocks, averaging each 7 x 7 feature map into a single number throws away where in the image each pattern is. Position matters for digits, because a 1 and a 7 share a vertical stroke and differ only in where the other strokes are. This control tested a head that was too weak, so it could not answer the size question, and the size-matched control was added instead.

The improvements. BatchNorm and dropout reduced the errors from 44 to 30 and the lowest validation loss from 0.0433 to 0.0295. The plain CNN reached 0.998 training accuracy while its validation loss rose after epoch 6, which is overfitting (Figure 1). Model B ends with training accuracy (0.990) close to validation accuracy (0.992), so the overfitting gap is closed. B, C and D make 30, 31 and 28 errors, so augmentation and the learning-rate schedule show no measurable gain on validation. They still matter for the final choice. At its last epoch, D has the highest validation accuracy of the three (0.9929 against 0.9921 for B and 0.9905 for C) and the lowest validation loss (0.0262 against 0.0308 and 0.0306). The cosine schedule brings the learning rate close to zero at the end, so the weights settle instead of jumping around. The final models train on all the data and have to stop at a fixed epoch, so I chose D, the configuration that is in its best state at its last epoch.

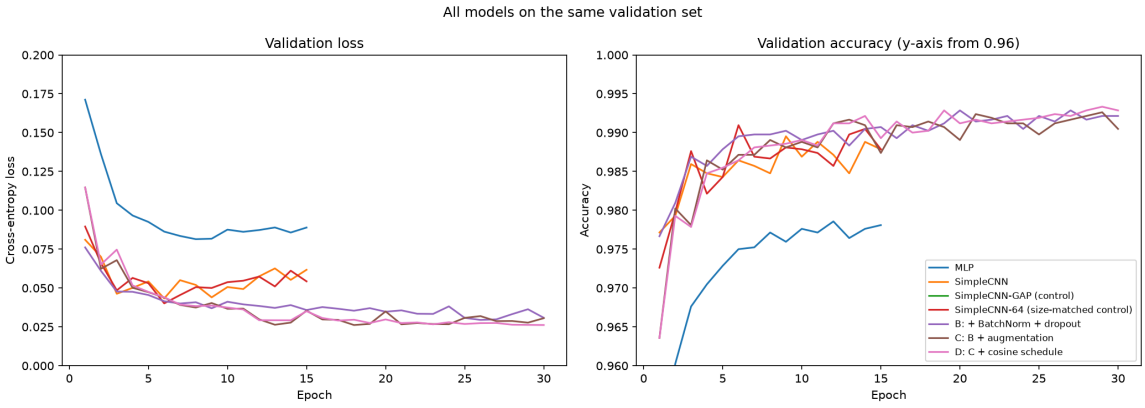


Figure 1. Validation loss (left) and accuracy (right) per epoch for all models on the same validation set. The models without regularisation reach their lowest loss around epochs 6 to 8 and then rise, while B, C and D keep a low loss for 30 epochs. The GAP control lies outside both plotted ranges (loss above 0.2, accuracy below 0.96).

Ensemble. The final ensemble cannot be validated, because it is trained on the validation images. As a check of the idea, I averaged the probabilities of B, C and D on the validation set. The average made 21 errors, against 28 to 31 for each model alone. That gain of 7 to 10 images is just inside the noise band, so it is consistent with ensembling helping but does not prove it. On the test set, each of the five final models disagrees with the ensemble on only 76 to 95 of the 28,000 images. The predicted share of every digit on the test set is within 0.4 points of its share in the training data, so the ensemble does not over- or under-predict any class.

Validation versus Kaggle. The Kaggle score (0.9923) is lower than the validation results suggested (0.9950 for the average of B, C and D). There are three likely reasons. The Kaggle score is itself a measurement on a sample of images, with its own uncertainty. Even on all 28,000 test images it would be about +- 0.1 points, and more if the public leaderboard uses only part of the test set, as many Kaggle competitions do. The validation results are slightly optimistic because validation was used to choose the model and the epoch (section 4). And the test set very likely contains mislabelled images, as the training data does.

Error analysis. The confusion matrix of model D (Figure 2) shows that its 28 validation errors are spread thinly. No confusion happens more than 4 times. 8 is the hardest digit (7 errors) and 6 the easiest (none). The most frequent confusions are 8 → 9 and 0 → 6, with 4 each. Both pairs share most of their shape. An 8 with a small lower loop looks like a 9, and a 0 whose stroke continues past the top of the loop looks like a 6.

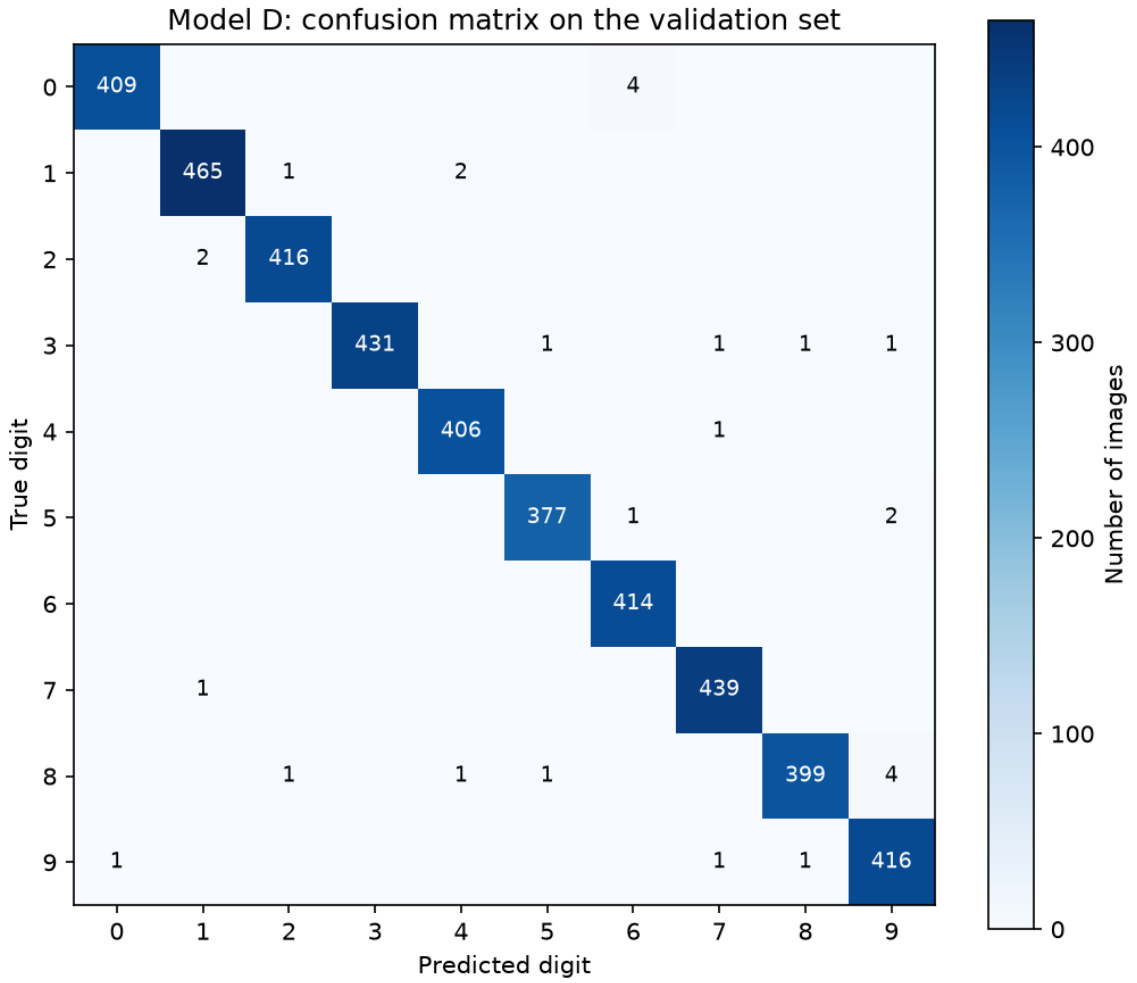


Figure 2. Confusion matrix of model D on the 4,200 validation images. Rows are the true digits and columns the predictions. Only 28 images lie off the diagonal.

I reviewed all 28 errors by eye (Figure 3) and sorted each into one category. The review is saved in `results/misclassified_review.csv` with each image's row in `train.csv`, so every judgement can be checked against the original data.

Category	Count
Mislabelled in the dataset	4
Genuinely ambiguous	7
0 versus 6 or 9 (stroke past the loop, or a very short stem)	4
Sloppy but readable	9
Clear model mistakes	4

Four of the errors are labelling errors in the dataset. For example, row 24,477 of `train.csv` is labelled 3 but clearly shows a 5, and the model predicted 5. In these cases the model is right and the label is

wrong. The mislabelled images are concentrated among the model's most confident mistakes. Four of the eight most confident errors are label errors. Only 4 of the 28 errors are clear model mistakes. The rest come from the data itself or from unusual handwriting. I did not correct or remove the mislabelled images. Relabelling them by my own judgement would change the data in a way the code cannot reproduce, and correcting validation labels would make the score look better than it is.

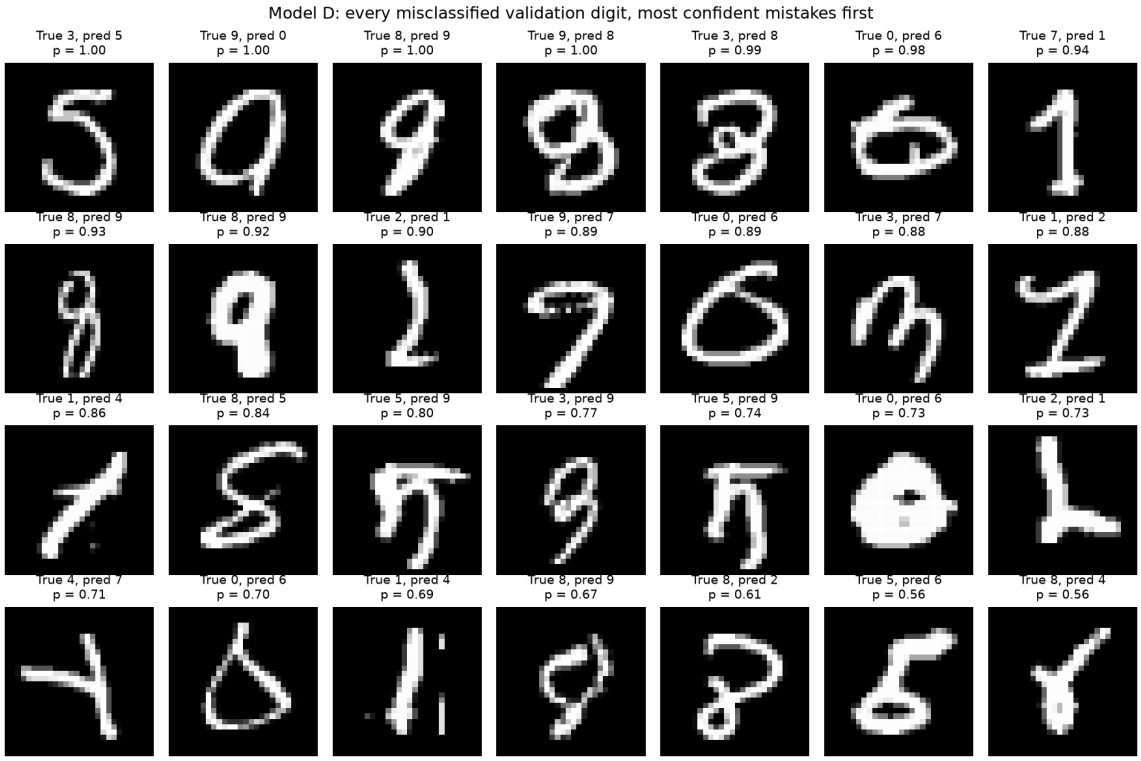


Figure 3. All 28 misclassified validation images of model D, sorted from the most to the least confident mistake. Each title gives the label in `train.csv`, the prediction and the model's probability for it. The first image is the 5 labelled as 3. The grid numbers in `misclassified_review.csv` count from the top left, row by row.

The notebook and the `results/` folder also contain the training curves of the MLP and the plain CNN, and the comparison table as a CSV file.

6. Conclusions and limitations

Answers to the three questions.

1. **The CNN improves substantially on the MLP.** It halves the validation errors (90 to 44) and reaches 0.9895 against the MLP's 0.9786.
2. **The improvement comes from the convolutions, not from size.** A CNN with fewer parameters than the MLP still makes less than half its errors (38 against 90). Locality and translation invariance let the CNN learn stroke patterns once and recognise them anywhere, while the MLP has to learn them separately for every position.
3. **BatchNorm and dropout fix the overfitting.** They reduce the errors from 44 to 30 and close the gap between training and validation accuracy. Augmentation and the cosine schedule give no

measurable gain on validation, but the schedule leaves the model in its best state at the last epoch. That is what made D the right choice for final models trained without a validation set.

The final ensemble scored **0.99232** on Kaggle. The manual review shows that only 4 of model D's 28 validation errors are clear model mistakes. The rest come from mislabelled images, digits that are ambiguous even to a person and unusual handwriting. At this level, the data limits the score more than the model does.

Limitations.

- **Single hold-out split.** All comparisons rest on one validation set of 4,200 images, with an uncertainty of about ±13 images. B, C and D cannot be separated, and the gain from plain CNN to B is only just above that threshold. Cross-validation or a paired test such as McNemar's test would make these comparisons more reliable.
- **Augmentation was not trained to convergence.** C and D still reached only about 0.97 training accuracy at epoch 30, below their validation accuracy, so they were still learning. Their result probably understates what augmentation can do. **A possible improvement is to train C and D for longer, for example 50 epochs.** That decision should be made on the validation set first. Only a clear gain there, more than the noise band, would justify retraining the final ensemble and resubmitting. I did not run this because of the training time (about 1 to 1.5 hours on my laptop).
- **The ensemble is not validated directly.** It is trained on all the data, so its benefit is supported only indirectly, by averaging B, C and D on validation (21 errors against 28 to 31). That gain is inside the noise band.
- **Validation scores are slightly optimistic.** Validation was used to choose the epoch and the model, so the best of many noisy values was picked each time. This is one reason the Kaggle score (0.9923) is below the validation results.
- **Mislabelled images were left in the data.** They add noise to training and make the validation and test scores lower than the model's true accuracy. Removing them reproducibly would need an automatic method, for example flagging images where several independent models confidently disagree with the label, instead of my own judgement.
- **Few hyperparameters were tuned.** Dropout rates, learning rate, batch size and network width were set once and not searched. A small search on the validation set might find a better configuration, but with the same risk of selecting on noise.
- **The GAP control only tested a shallow network.** Global average pooling failed here because two convolutional blocks leave each feature map with too little information. Deeper networks such as ResNet use it successfully, which is why their classifier head is so small (Session 12 notebook, section 2). This result should not be read as evidence against global average pooling in general.

7. AI use

I used Claude for feedback on the code I wrote, helped with writing the report ie. clarifying ideas and spell checking.